Mastering Trinamic Stepper Motor Drivers

By Daniel Frenkel

Contents

Introduction *4*
Prior Knowledge 4
A Bit About Me 4
Required Hardware *6*
Selecting your microcontroller **6**
A note on the ESP32 7
Hobby vs Pro 8
Trinamic TMC2209 ..... *8*
Breadboard Setup *10*
Schematic **10**
Arduino Setup *11*
Installing ESP Core 11
Installing Libraries 11
Stepper Motor Basics *11*
Power Requirements *12*
Current Sense Resistors **13**
RMS Current **14**
Voltage **15**
Understanding Trinamic Drivers *16*
The STEP and DIR Pins **17**
Modes of Operation **17**
Examples **18**
Example 1: Dual Core 19
Changing Settings 19
What is a register? 19
Sending Datagrams 24
TMCStepper Library *24*
Multiple Motors 24
Single Motor Operation 25
Documentation 26
FastAccelStepper *29*
FastAccelStepper vs AccelStepper 29
Dual Core Setup **30**
Motor Setup **30**
Understanding Positioning **33**
Starting a Movement **34**

Advanced Features *36*

StallGuard **36**

Overview 36

StallGuard Setup 38

Setting StallGuard Values 38

How SG_RESULT works 39

CoolStep **44**

Finite State Machine **45**

Single-Core Operation **45**

Conclusion *48*

Introduction

All code and reference materials will have been emailed to you after purchasing this guide.

In this guide, I will teach you to program and run a stepper motor in Arduino using a Trinamic stepper motor driver. My goal is to make this simple so that someone completely new will be able to learn. That is why we will use Arduino.

Although we will use the TMC2209, most Trinamic drivers work the same way so you can apply this knowledge to other drivers as well.

Prior Knowledge

It's best to read this book if you already have some knowledge about basic electronics and basic Arduino programming. If you don't, that's ok, just spend more time with Google and YouTube when you encounter something that you don't understand. If you'd like help with your project, I am always happy to help consult you on it. It's always best to do things right early on in the project, so please don't ever hesitate to reach out to me at danielfrenkel@gmail.com.

A Bit About Me

I know you're eager to get started so I'll make this section short. Like you, I'm a maker. I love to design and build things, mostly things that will make my life easier. I started my own electronics journey in 2017 with Arduino when I decided to build a motorized device that would open my sliding glass door remotely. I wanted a way to let my dogs out of the house by using Alexa to open and close the door. I also wanted the door to automatically close because my wife kept leaving the door open and there's nothing more painful than watching that cool air-conditioned air leave the home in the summer. Here's a photo of the device, disassembled while I was trying to fix something. I used an ESP32 and TMC5160 with a massive NEMA 34 motor for this project and it worked incredibly well.

©2024 Daniel Frenkel



In early 2020, I posted this project online and was contacted by Tyler Schrenk who wanted me to retrofit this device onto his sliding window. In his early 20s, Tyler dove head-first into a lake he had assumed to be shallow and ended up damaging his spinal cord and became quadriplegic as a result. He was always looking for ways to automate his home and make his life a bit easier. He was also very involved in helping different organizations create new assistive technologies, such as working with the University of Washington's Personal Robotics Lab, and on Microsoft's adaptive gaming controllers.

I decided to help Tyler and got to building a window opener. This is version 1 of the device:



With this prototype, I had a device that worked well and managed to sell a handful of them as kits. However, it had many problems that needed to be addressed. The biggest problem was creating a design that could be attached to all types of windows. Some windows require very little force to move, and others require a massive amount. Some windows have windowsills, others don't. The biggest hurdle has been creating a device that universally fits all windows.

©2024 Daniel Frenkel

To help fix these problems, I spent two full months writing an SBIR grant through the United States Department of Health and Human Services, and I got the grant! Tragically, due to a lung infection, Tyler died just one month after receiving the grant. But my work on the window opener continues as of this writing because I know Tyler would have wanted others to have it.

Unfortunately, I cannot post any photos or many details of the device because it uses a lot of proprietary technology that's taken me years to figure out. But it's a very advanced piece of hardware and many of the problems have been addressed. I'm in the process of applying for Phase 2 of the grant which will give us the resources to finally release the final product.

Thank you for allowing me to share the knowledge that I've obtained over the many years of working with Trinamic stepper motors.

Required Hardware

Selecting your microcontroller

Any microcontroller will work to drive the stepper driver. But please be aware of the limitations. There are two ways to use the TMC2209, you can use the STEP/DIR interface or use the built-in Pulse Generator. Please read this from the datasheet, which is critical to understand:

1.3 Moving and Controlling the Motor

1.3.1 STEP/DIR Interface

The motor is controlled by a step and direction input. Active edges on the STEP input can be rising edges or both rising and falling edges as controlled by a special mode bit (DEDGE). Using both edges cuts the toggle rate of the STEP signal in half, which is useful for communication over slow interfaces such as optically isolated interfaces. The state sampled from the DIR input upon an active STEP edge determines whether to step forward or back. Each step can be a fullstep or a microstep, in which there are 2, 4, 8, 16, 32, 64, 128, or 256 microsteps per fullstep. A step impulse with a low state on DIR increases the microstep counter and a high decreases the counter by an amount controlled by the microstep resolution. An internal table translates the counter value into the sine and cosine values which control the motor current for microstepping.



1.3.2 Internal Step Pulse Generator

Some applications do not require a precisely co-ordinate motion – the motor just is required to move for a certain time and at a certain velocity. The TMC2209 comes with an internal pulse generator for these applications: Just provide the velocity via UART interface to move the motor. The velocity sign automatically controls the direction of the motion. However, the pulse generator does not integrate a ramping function. Motion at higher velocities will require ramping up and ramping down the velocity value via software.

STEP/DIR mode and internal pulse generator mode can be mixed in an application!

If using the STEP/DIR interface, your microcontroller will require 100% of its processing to pulse the STEP pin, which means you cannot do anything else while it moves if you want smooth motion. If other tasks are run during a movement, there will be a noticeable stutter in the motor as the IC switches between your other task and pulsing the STEP pin. For example, if you want to read the StallGuard data while moving the motor, it will not work very well and setting up StallGuard will be difficult on a single-core IC.

That is why using the internal Step Pulse Generator is a great idea with a single-core processor. In this guide, I will use the ESP32 because it has two cores. This allows us to run the motor on one core, and read various UART values from the TMC2209 while setting it up.

A simple Arduino UNO will work completely fine in STEP mode, but again, will be difficult to tune StallGuard on. Please note that included are multiple examples that use both the STEP/DIR and Pulse Generator, this way you can use whatever processor you prefer.

This guide will use an ESP32 because it has two cores.

A note on the ESP32

There are many versions of the ESP32 now available (About 60 as of my last count). The original ESP32 was released in 2016 and has quickly become one of the most popular microprocessors on the market due to its rich feature set, low cost, and wireless capabilities.

The ESP32 can be divided into four categories, with several subcategories:

1. **ESP32 Series**
2. **ESP32-S Series**
    a. ESP32-S2
    b. ESP32-S3
3. **ESP32-C Series**
    a. ESP32-C3
    b. ESP32-C6
4. **ESP32-H Series**

Each of the above series has advantages and disadvantages and choosing the right one will depend on your needs. The biggest difference that matters to us is the number of cores in each series. The first part of this guide will require an ESP32 with two cores. The second part of the guide will only require a single core processor. I recommend using two cores to really learn how all of this works.

Dual Core

- ESP32
- ESP32-S2
- ESP32-S3

Single Core

- ESP32-C3
- ESP32-C6
- ESP32-H2

Hobby vs Pro

There are typically two groups of users, hobbyists and professionals. Typically, hobbyists are concerned with keeping costs low, and professionals are concerned with keeping their headaches low. For this reason, I have created two sections for hardware requirements.

Hobby Hardware:

1. Espressif ESP32-DevKitC-32E Development Board
2. BIGTREETECH TMC2209 V1.3
3. Stepper Motor
   a. The size of the motor does not matter for this guide. I prefer the NEMA 17 size rated for 2A but almost any hybrid stepper motor will work.
4. Jumper Wires
5. Breadboard
6. 5.5V - 28V 2A+ Power Supply
   a. Depending on your requirements, you can use a very wide range of power supplies. Please see the Power Requirements section to learn more.

Professional Hardware:

1. Espressif ESP32-DevKitC-32E Development Board
2. TMC2226-BOB or TMC2209-BOB
   a. I prefer the TMC2226 because its features are identical to the TMC2209. But due to its larger footprint, it can handle more current without overheating.
3. Stepper motor from Lin Engineering.
   a. While these motors cost almost ten times more than one from Amazon, the access to the motors torque curve and 3D STEP files is invaluable. Their quality control is also very good, so you know exactly what you are getting every time.
   b. Be sure to find a motor that meets your speed and torque requirements.
4. Jumper Wires
5. Breadboard
6. 5.5V - 28V 2A+ Power Supply
   a. Depending on your requirements, you can use a very wide range of power supplies. Please see the Power Requirements section to learn more.

Trinamic TMC2209

NOTE: Although we will use the TMC2209, this course will work with any Trinamic stepper motor driver as they function very similarly. However, this guide will not work with their line of motion controllers like the TMC5130.

We will be using the BIGTREETECH TMC2209 V1.3 module that can be found on Amazon and AliExpress, links are on the GitHub page for the correct version.

The TMC2226-BOB and TMC2209-BOB will work just the same as the BIGTREETECH. They are just significantly more reliable because they are made by Trinamic, and StallGuard is significantly easier to use since they have a built-in LED when stalling.

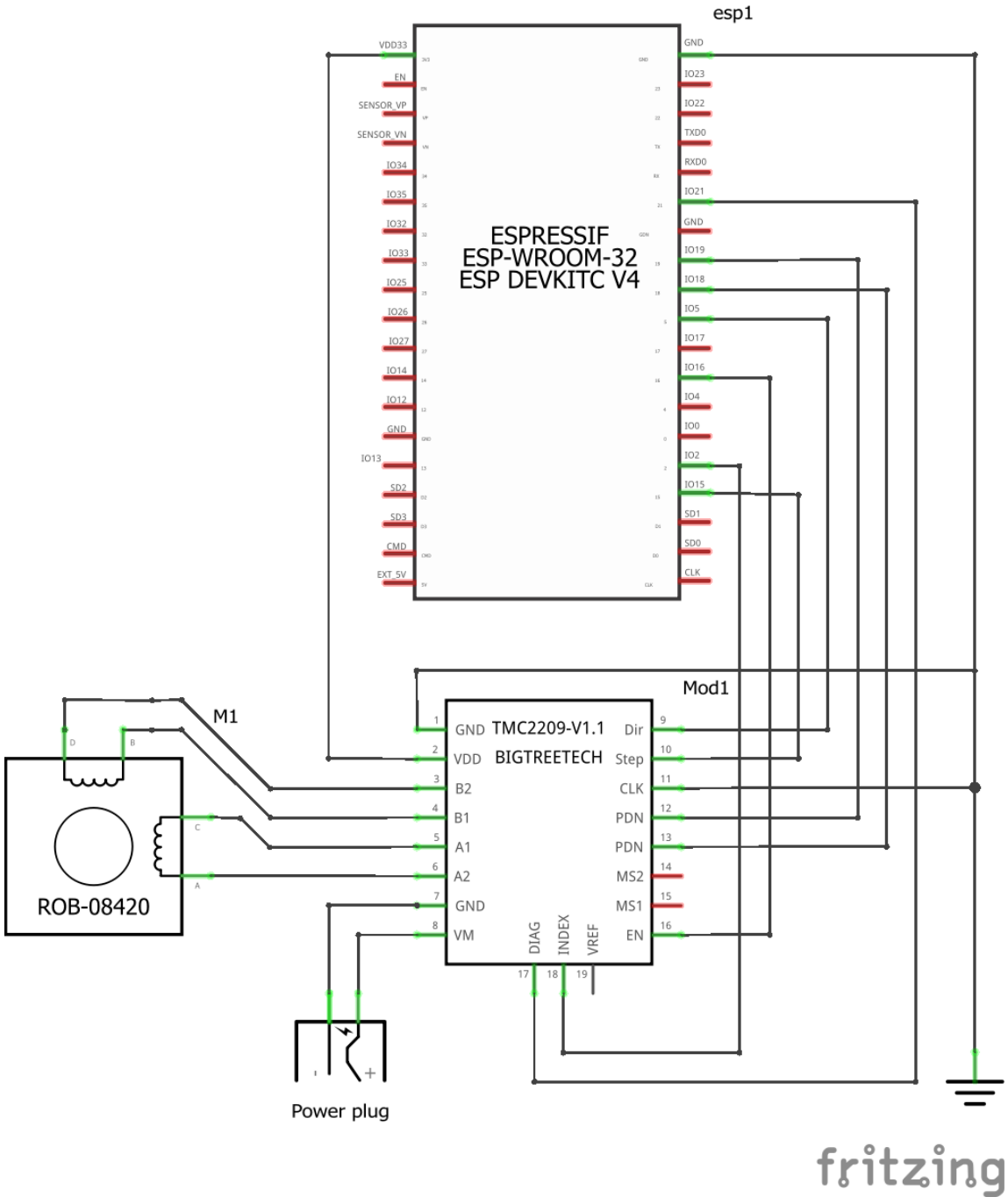
It is very important that you purchase the V1.3 of the BIGTREETECH boards and not the older version because the older versions do not give us access to the TX and RX pins. This specific module was designed to use on 3D printers. While this board is designed to be used in 3D printers, it's cheaper and more available than the official breakout board from Trinamic, which is why this guide will use it.

©2024 Daniel Frenkel

Breadboard Setup

Note the StallGuard pin (DIAG) is sticking up in the air.

Schematic



Arduino Setup

Now that we know how a stepper motor works, we need to program our ESP32 to make it move. I have taken the time to write all the firmware myself. What we will do in this course is examine it line by line, so you understand how each part works.

Let’s open the firmware in Arduino to begin. Open the included dual_core folder. Open dual_core_simple.ino in the Arduino IDE 2.

Installing ESP Core

Follow this guide to install the ESP32 board.

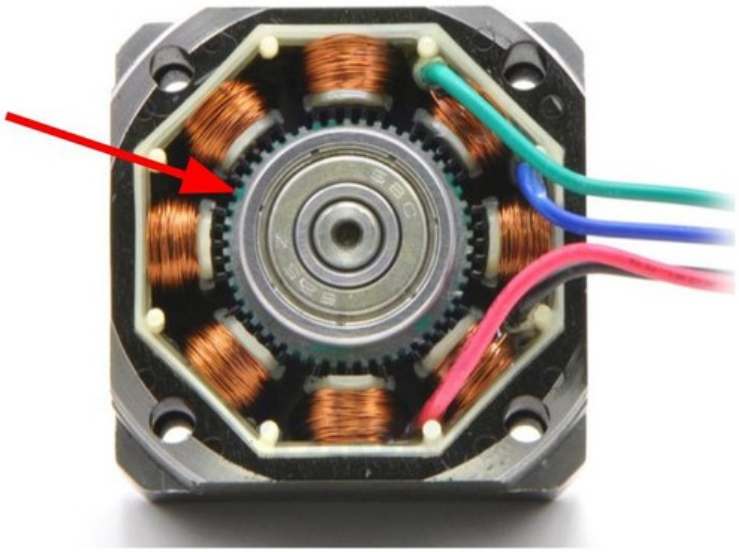
Installing Libraries

Before we begin, we'll need to install some libraries to use this sketch. Open the Arduino library manager and install the following libraries

1. TMCStepper
2. FastAccelStepper

Stepper Motor Basics

Let's quickly cover some basic information about stepper motors. I won’t go deep here and will skim over what needs to be understood later in this course. I recommend watching this quick video that you can find the link to on the resource page.



The most important thing to understand is that the average stepper motor has 50 physical teeth around the rotor.

The term "Full Step" means that the motor moves 1/4 of these 50 steps at a time. If it turns 200 full steps, the motor will have completed 1 full revolution.

However, turning a stepper motor in full step mode is rather violent. The motor vibrates excessively and is very noisy in the process. That is why the TMC2209 does not even allow us to use full step mode. It automatically translates one full step into 256 microsteps, making motor operations extremely smooth.

The term "Microsteps" is when the motor driver moves a fraction of a full step. Imagine slicing one of those 200 full steps into 256 small parts. The motor can be made to stop at any of those exact microsteps.

With 256 microsteps, we can divide each revolution in 51,200 steps! That's because if we have 200 steps, and each step has 256 microsteps, that is 200 x 256 = 51,200!

Power Requirements

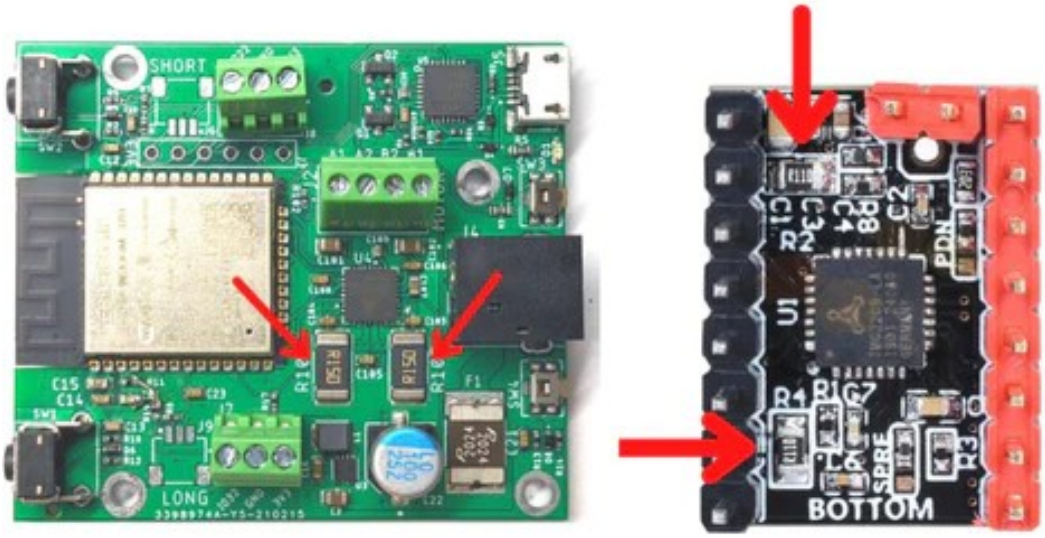
Understanding what size power supply you need is very important. In this section we'll cover the power requirements of the TMC2209 so that you apply the correct voltage and size your power supply correctly.

Current Sense Resistors

The TMC is always checking how much current is passing through it. Otherwise, it will not be able to accurately output the current that you told it to in the settings. The way it checks its current is by use two current sense resistors that are placed near the driver.

The driver sends its entire current through the resistors which is why they are so large. It then reads back the voltage and can figure out how much current it is using at any moment in time. For more info on how Current Sense Resistors work, please Google it.

What matters most is the value of the two resistors that were placed near the driver, because the value will determine the maximum current the driver can use. Note that the engineer who designed your board could have chosen any value they wanted. But to properly run your code, we need to know this value.



Let's go to page 49 of the datasheet and look at this table:

CHOICE OF R_{SENSE} AND RESULTING MAX. MOTOR CURRENT		
R_{SENSE} [Ω]	**RMS current [A]** VREF=2.5V (or open), *IRUN*=31, *vsense*=0 (standard)	**Fitting motor type** (examples)
1.00	0.23	
0.82	0.27	
0.75	0.30	300mA motor
0.68	0.33	400mA motor
0.50	0.44	
470m	0.47	500mA motor
390m	0.56	600mA motor
330m	0.66	700mA motor
270m	0.79	800mA motor
220m	0.96	1A motor
180m	1.15	1.2A motor
150m	1.35	1.5A motor
120m	1.64	1.7A motor
100m	1.92	2A motor
75m	2.4*)	

Let's see what happens when 0.10 Ohm resistors are used. Looking at the chart, it means that the max current that can be used by this board is 1.92A RMS.

That is what the value of 0.10f (the "f" is for floating point number) is for, because we have a 0.10 Ohm resistor. Be sure to check your board to find the value of the resistors. It should be listed somewhere on the datasheet, otherwise it will most likely be stamped onto the resistors themselves.

Quick note on power. While the TMC2209 states it can handle 2A, don't be fooled. The TMC2209 is very small and gets extremely hot, extremely fast. That's why I prefer the TMC2226, because it's much larger and has a much larger ground pad which helps dissipate heat faster. Just be sure that you use a massive ground plan on your custom PCB and add a heat sink and maybe a fan if you plan to run it on high power for extended periods.

Fortunately, there is an internal temperature sensor, and you can monitor when it's about to overheat via the DIAG and INDEX pins (See section 15.4 of the datasheet). If it does overheat, it will simply shut off and start working when it cools down. Sometimes this will result in your motor randomly starting/stopping during a movement.

RMS Current

RMS current is important to understand because it determines what size power supply we need. If our power supply is too small, it may overheat and start a fire.

RMS means Root Mean Square. It is the average current the motor is using. Sometimes it's using less, sometimes it is more. For example, when the motor needs to push a heavy object, its current draw will increase.

To determine max current, take the RMS and multiply it by 1.41. Why 1.41? Because Trinamic says so.

RMS x 1.41 = Max Current

If our RMS is 1.92A, our max current will be 1.92A x 1.41 = 2.71A

Since the ESP32 requires at least a 500mA power supply as stated in the ESP32 datasheet, this will give us a max current draw of 2.71A + 0.5A = 3.21A

We need to purchase at least a 3.21A power supply to make sure it does not get overloaded or overheat if you want to run the device at max power. Of course, you can choose to keep the current low in the firmware and use a smaller power supply.

Voltage

While we're here, let's discuss voltage as it's a big source of confusion when it applies to stepper motors. If we go back to our electronics 101 course, we probably learned that current is like the volume of water that the motor pulls while running. And voltage is the force of the water that gets pushed into the motor.

When it comes to stepper motors, we need to consider the voltage that is going into the driver and not the motor itself. Do not pay attention to the voltage rating of the stepper motor, only pay attention to the voltage range that the driver accepts.

For the TMC2209, the range is 4.75 to 29V. The reason the range is so wide is because voltage is only a concern when we want to increase the speed of the motor. The higher the voltage, the higher the maximum speed for the motor.

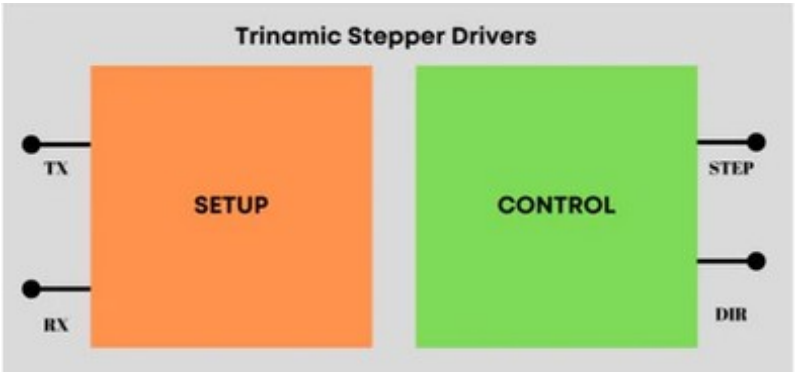
The higher the voltage, the faster the current in the windings will reach its new target value from one step to the next, overcoming the inherent inductance value of the winding and building the magnetic field in the rotor. So, a higher voltage will result in better speed performance.

To have full torque, we need to increase the voltage as we speed up the motor. If we run the motor very slowly, we can achieve full torque at a low voltage. You can speed the motor up, but it will become weaker as it spins faster, unless you increase the voltage to the driver.

©2024 Daniel Frenkel

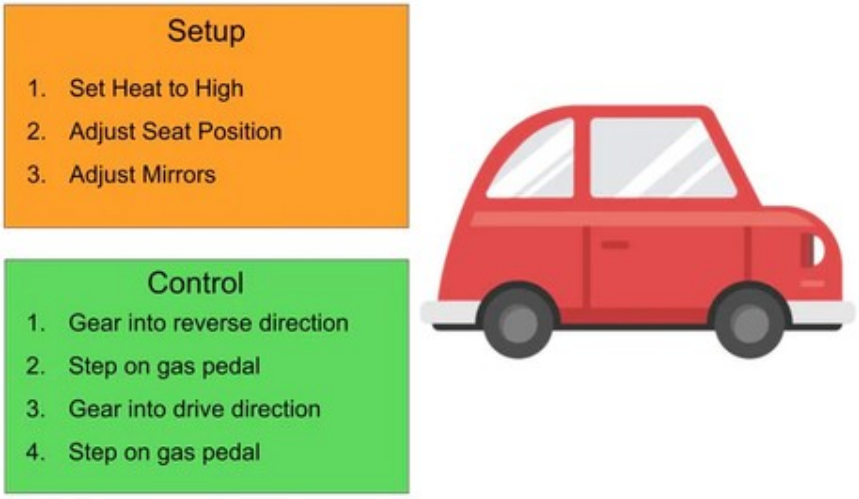
Understanding Trinamic Drivers

Trinamic drivers need to be thought of in two separate parts: **Setup** and **Control**.



Using SPI or UART (depending on the model of driver you have), we can set dozens of different settings. Then we control the driver using the Step and Dir pin to make out motor turn.

Always try to think of Trinamic drivers as these two very different parts, Setup and Control.



Let me give you an example of setup and control. Let's pretend you want to drive your car to the restaurant. What do you?

You get in your car and turn the engine on. Then you change the heat settings to make it hotter or colder. Then you adjust the seat to make sure it feels comfortable and maybe adjust the radio and prepare for the drive. Then you place your car into the drive gear and press the gas pedal to drive forward.

In this scenario you first setup the car by changing all the setting prior to driving. Then you placed it into gear and pressed the gas pedal to control it.

©2024 Daniel Frenkel



Trinamic drivers work in a similar way. We use UART or SPI (depending on the driver you use) to set different settings in the driver before we move it. We can set the current, stall value, and many more things using the TMCStepper library.

Then we pulse the STEP and DIR pin to get the motor to begin turning by using the FastAccelStepper Library.

The STEP and DIR Pins

Making the motor turn requires just two pins on almost any stepper driver, even primitive ones like the A4988. These are the Step pin and the Dir pin. The Dir pin is the Direction pin. When it gets a high signal, it spins one way, and when it gets a low signal, it spins the other way.

The Step pin as also very simple. When it receives a high signal from the ESP32, it moves one step. It's that simple. If we want it to accelerate or move at a high speed, we just change the rate of high signals that we send from the ESP32. Think of it like flipping a light switch, each time you flip the switch on and off, the motor moved one step. If you want it to mover very fast, you just flip it on and off very very fast.

Modes of Operation

The TMC2209 was designed to replace older stepper motor drivers in 3D printers. To make it compatible with a wide range of 3D printers as well as advanced users like us, they created

three different modes of operation. This is important to understand because reading the datasheet will get confusing without understanding this.

> **NOTE:** The Trinamic datasheet is **VERY IMPORTANT**. It contains so much information and is critical to understand. Take your time to read it carefully when required.

Let's see kinds of settings there are for us to use. Open the TMC2209 datasheet by downloading it from the resources page or the Trinamic website.

Let’s go to page 4, at the very bottom section titled Three Modes of Operation:

OPTION 1: Standalone STEP/DIR Driver (Legacy Mode): This mode is designed so that boards like the BIGTREETECH 2209 V1.2 fit perfectly in place of older 3D printer drivers. This is called legacy mode and is very crude. We will not use it.

OPTION 2: Standalone STEP/DIR Driver with OTP pre-configuration: I am not sure what this mode is for. I think it may be for newer 3D printers, but we will not use this either.

OPTION 3: STEP/DIR Driver with Full Diagnostics and Control: This gives us full control of all settings via UART. This is what we will use.

So, what kind of settings can set via UART? One of the most important settings is the current setting. We can set the current of our motor to use very little current which will allow it to run cooler and safer, or we can use a lot of current to move very heavy objects. This is an example of just one of dozens of settings we will control via UART, something that is not possible to do in Legacy Mode (Options 1).

Examples

The best way to learn is to dive right in. We’ll open the example and learn it as we go. If you only want to learn to use the Single Core code, please read this section as well because it will cover many important things that will need to be considered for the single core version, because single core operation is more complicated.

Example 1: Dual Core

First, we include the two libraries that we'll need.
It's completely possible to write your code without the aid of these two libraries. The

```
#include <TMCStepper.h>
#include <FastAccelStepper.h>
```

TMCStepper library helps us set up the TMC2209 settings. And the FastAccelStepper library pulses the STEP/DIR pins and keeps tracks of steps for us, so that we don't have to.

First, let's talk about the settings that are possible to set on the TMC2209 and why using a library is extremely helpful in setting them.

Changing Settings

How do we change the stall, current, or any of the other 50+ settings? To change the settings on the driver, we just need to change the value of specific registers. Think of the driver as an airplane cockpit. There are a million switches everywhere. All we need to do is flip the correct switch, which the datasheet tells us how to do.

What is a register?

A register a small computer circuit that can hold a small piece of information. Think of it like a small box that contains a group of on/off switches, like a light switch. Each of these light switches controls a different setting depending on its state of on or off. It's our job to find which box to open, and which switch to flip to change a setting.

This next section might be a little dense, but it's important to understand. Normally we will not program the driver in this way because the TMCStepper library abstracts this step and does it all for us. But it's very important to understand how things work under the hood. Without this knowledge, you will not understand how the datasheet works and will struggle because of it.

Go to page 19 of the datasheet titled "Register Map" This is the beginning of the section that shows us which registers we can set. Let's look around as this is the most important and interesting section in the datasheet.

Here we have a quick overview of the Register Map section. There are typically three types of registers. Ones that you can only read data from, ones you can only write data to, and ones that allow you to do both.

1. **R** means you can read the register.
2. **W** means you can write to the register.
3. **R/W** means you can read and write to the register.

©2024 Daniel Frenkel

5 Register Map



This chapter gives an overview of the complete register set. Some of the registers bundling a number of single bits are detailed in extra tables. The functional practical application of the settings is detailed in dedicated chapters.

Note
- *Reset default*: All registers become reset to 0 upon power up, unless otherwise noted.
- Add 0x80 to the address **Addr** for write accesses!

NOTATION OF HEXADECIMAL AND BINARY NUMBERS	
0x	precedes a hexadecimal number, e.g. 0x04
%	precedes a multi-bit binary number, e.g. %100

NOTATION OF R/W FIELD	
R	Read only
W	Write only
R/W	Read- and writable register

Let's explore our first register. On page 20 we see one of the most important registers called GCONF which is the first of our General Configuration Registers.

5.1 General Registers

GENERAL CONFIGURATION REGISTERS (0x00...0x0F)					
R/W	**Addr**	**n**	***Register***	**Description */ bit names***	
				Bit	***GCONF* – Global configuration flags**
RW	0x00	10	*GCONF*	0	*I_scale_analog* (*Reset default*=1) 0: Use internal reference derived from 5VOUT 1: Use voltage supplied to VREF as current reference
				1	*internal_Rsense* (*Reset default: OTP*) 0: Operation with external sense resistors 1: Internal sense resistors. Use current supplied into VREF as reference for internal sense resistor. VREF pin internally is driven to GND in this mode.
				2	*en_SpreadCycle* (*Reset default: OTP*) 0: StealthChop PWM mode enabled (depending on velocity thresholds). Initially switch from off to on state while in stand still, only. 1: SpreadCycle mode enabled A high level on the pin SPREAD inverts this flag to switch between both chopper modes.
				3	*shaft* 1: Inverse motor direction
				4	*index_otpw* 0: INDEX shows the first microstep position of sequencer 1: INDEX pin outputs overtemperature prewarning flag (*otpw*) instead
				5	*index_step* 0: INDEX output as selected by *index_otpw* 1: INDEX output shows step pulses from internal pulse generator (toggle upon each step)
				6	*pdn_disable* 0: PDN_UART controls standstill current reduction 1: PDN_UART input function disabled. Set this bit, when using the UART interface!
				7	*mstep_reg_select* 0: Microstep resolution selected by pins MS1, MS2 1: Microstep resolution selected by MSTEP register
				8	*multistep_filt* (*Reset default*=1) 0: No filtering of STEP pulses 1: Software pulse generator optimization enabled when fullstep frequency > 750Hz (roughly). TSTEP shows filtered step time values when active.
				9	*test_mode* 0: Normal operation 1: Enable analog test output on pin ENN (pull down resistor off), ENN treated as enabled. *IHOLD*[1..0] selects the function of DCO: 0...2: T120, DAC, VDDH *Attention: Not for user, set to 0 for normal operation!*

GENERAL CONFIGURATION REGISTERS (0X00...0X0F)					
R/W	Addr	n	Register	Description / bit names	
				Bit	GCONF - Global configuration flags
				0	I_scale_analog (Reset default=1) 0: Use internal reference derived from 5VOUT 1: Use voltage supplied to VREF as current reference

Let's begin by reviewing the different columns of these registers.

On the left in green is the column R/W, which tells us if we can Read, Write, or Read & Write this register.

To the right in red is the "Addr." column. This is the address of the register. Think of this as the name of the register box. When we want to write to a register, this is the address that we write to.

To the right in orange is the "n" column, this is the number of bits in the register. This is the number of switches that it has. In this case, the GCONF register is 10 bits (0-9) which means we can flip 10 switches on or off.

If you look at the purple "Bit" column, you can see all 10 bits (0-9) and what each bit controls. The Register name is just a name, we don't use it for anything except to make it easier for us to read.

Now let's look at the bits in the GCONF register. Here we have bits 0-9 allowing us to set various settings. These bits are all very random and do not have anything to do with each other. These are just different settings we can set. If, for example, we want to inverse the motor direction, we set bit 3 to the binary value of 1, which is like flipping the switch on.

Let's say we want to inverse the shaft direction. In that case, we need to set this entire GCONF register to a certain value. How do we set it? Well, it's easy. We send the integer value "8" to address 0x00. Doing so will inverse the motor direction.

Do you see why it's the number 8? Think of all these 10 bits as binary like the red numbers I placed on the datasheet below.

GENERAL CONFIGURATION REGISTERS (0x00…0x0F)					
R/W	Addr	n	Register	Bit	Description / bit names
					GCONF – Global configuration flags
			1	0	I_scale_analog (Reset default=1) 0: Use internal reference derived from 5VOUT 1: Use voltage supplied to VREF as current reference
			2	1	internal_Rsense (Reset default: OTP) 0: Operation with external sense resistors 1: Internal sense resistors. Use current supplied into VREF as reference for internal sense resistor. VREF pin internally is driven to GND in this mode.
			4	2	en_SpreadCycle (Reset default: OTP) 0: StealthChop PWM mode enabled (depending on velocity thresholds). Initially switch from off to on state while in stand still, only. 1: SpreadCycle mode enabled A high level on the pin SPREAD inverts this flag to switch between both chopper modes.
			8	3	shaft 1: Inverse motor direction
			16	4	index_otpw 0: INDEX shows the first microstep position of sequencer 1: INDEX pin outputs overtemperature prewarning flag (otpw) instead
RW	0x00	10	GCONF	5	index_step 0: INDEX output as selected by index_otpw 1: INDEX output shows step pulses from internal pulse generator (toggle upon each step)
			32	6	pdn_disable 0: PDN_UART controls standstill current reduction 1: PDN_UART input function disabled. Set this bit, when using the UART interface!
			64	7	mstep_reg_select 0: Microstep resolution selected by pins MS1, MS2 1: Microstep resolution selected by MSTEP register
			128	8	multistep_filt (Reset default=1) 0: No filtering of STEP pulses 1: Software pulse generator optimization enabled when fullstep frequency > 750Hz (roughly). TSTEP shows filtered step time values when active.
			256	9	test_mode 0: Normal operation 1: Enable analog test output on pin ENN (pull down resistor off), ENN treated as enabled. IHOLD[1..0] selects the function of DCO: 0…2: T120, DAC, VDDH Attention: Not for user, set to 0 for normal operation!
			512		

If we send the value 8, it will turn the shaft bit 3 into a value "1". This will inverse the motor shaft, which means the motor will spin in the other direction. This can also be accomplished by sending the binary value of "1000", which is also the integer "8".

What if we also want to turn on test_mode, which is bit 9, along with inverse the shaft? We send the value 520. That is because bit 9 is value 512, and bit 3 is value 8. If we send 520, it will turn the value of bits 3 and 9 into a 1.

See how simple this is. Just find the setting you want to change, find the address, and find the bit to set. Now that we know what address to send to (0x00) and what value (520), how do we send it in our code?

©2024 Daniel Frenkel

Sending Datagrams

Go to page 15 of the datasheet. In section 4.1.1 we see what needs to be done to send the value of 520 to address 0x00.

We need to send a 64-bit ‘datagram’ value using UART in Arduino. If we send the 64-bit number according to these instructions, the TMC2209 registers will get set. But this process can be complicated. Trinamic does offer an API that will help you out quite a bit on their GitHub page.

Luckily there's a very easy way. **The TMCStepper Library!**

TMCStepper Library

The TMCStepper library is an Arduino library written by Github user **teemuatlut**. It allows you to easily change the values of almost any Trinamic driver in a very easy way. I recommend using it for all Trinamic projects.

Let's go back to our Arduino sketch.

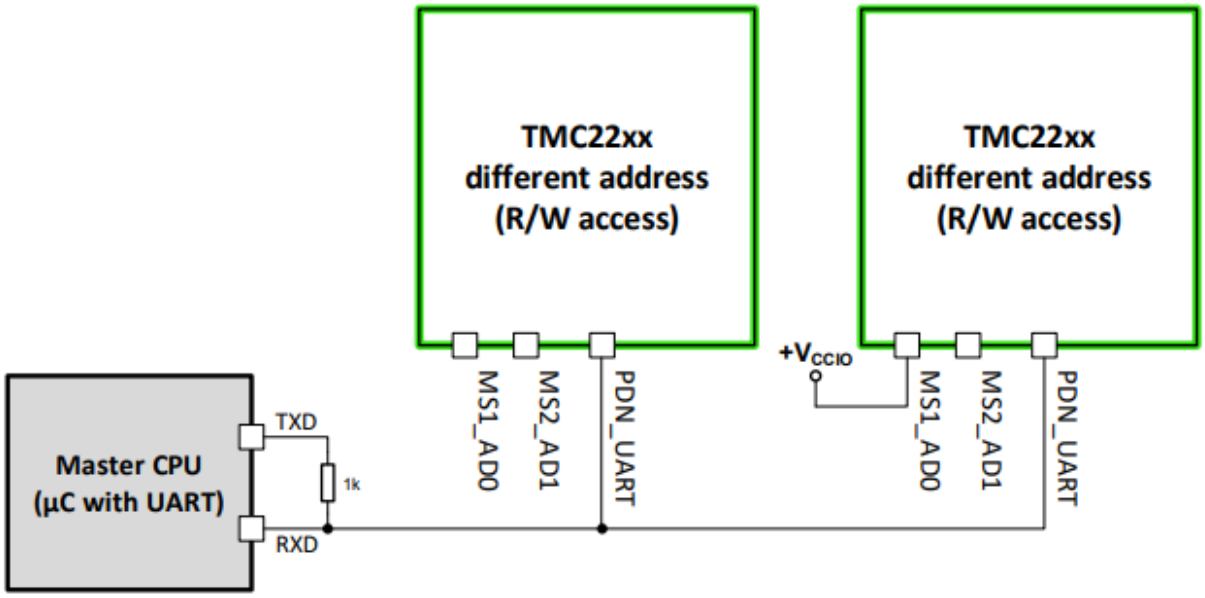
We’ll create an object for the TMCStepper class and pass the parameters required by the class. In this case, we will call it “driver”.

```
TMCStepper driver(&Serial1, 0.12f, 0);
```

Multiple Motors

We can control up to four TMC2209 motors/boards. All we have to do is address them correctly. To assign an address to each board, simply pull the MS1/MS2 pins high according to the chart below. We will only use one motor, which is how each TMC2209 is already set to do. Just leave the MS1/MS2 pins floating (they have a built-in pulldown resistor), for single motor

operation.



MS1/MS2: Configuration of Microstep Resolution for Step Input			
MS2	**MS1**	**Microstep Setting**	**UART Address**
GND	GND	8 microsteps	0
GND	VCC_IO	32 microsteps (different to TMC2208!)	1
VCC_IO	GND	64 microsteps (different to TMC2208!)	2
VCC_IO	VCC_IO	16 microsteps	3

To control additional motors, just create additional objects and change the address of the motor from 0-3 like below.

```
TMCStepper driver1(&Serial1, 0.12f, 1);
```

Single Motor Operation

We are going to run one motor only, so let's go back to the code to continue.

Let's open the documentation for the TMCStepper library. A link can be found at the top of the Arduino sketch.

As we can see in the documentation, the object constructor needs 3 parameters when creating

```
TMC2209Stepper (Stream *SerialPort, float RS, uint8_t addr)
```

it.

1. Serial port
2. RS value
3. Address

To begin, we type in "TMC2209Stepper" like you see below in purple to access the TMC2209 Stepper class. Then we need to create an object name, in this case let's call it "driver" like you see below in red.

```
TMC2209Stepper driver(&Serial1, 0.12f, 0);
```

Then we select the serial port to use. The ESP32 has 3 hardware serial ports, 0, 1, and 2. But only 0 and 1 can be used. Serial0 is used by the USB port for the serial monitor, so we can't use it, which leaves us to use Serial1.

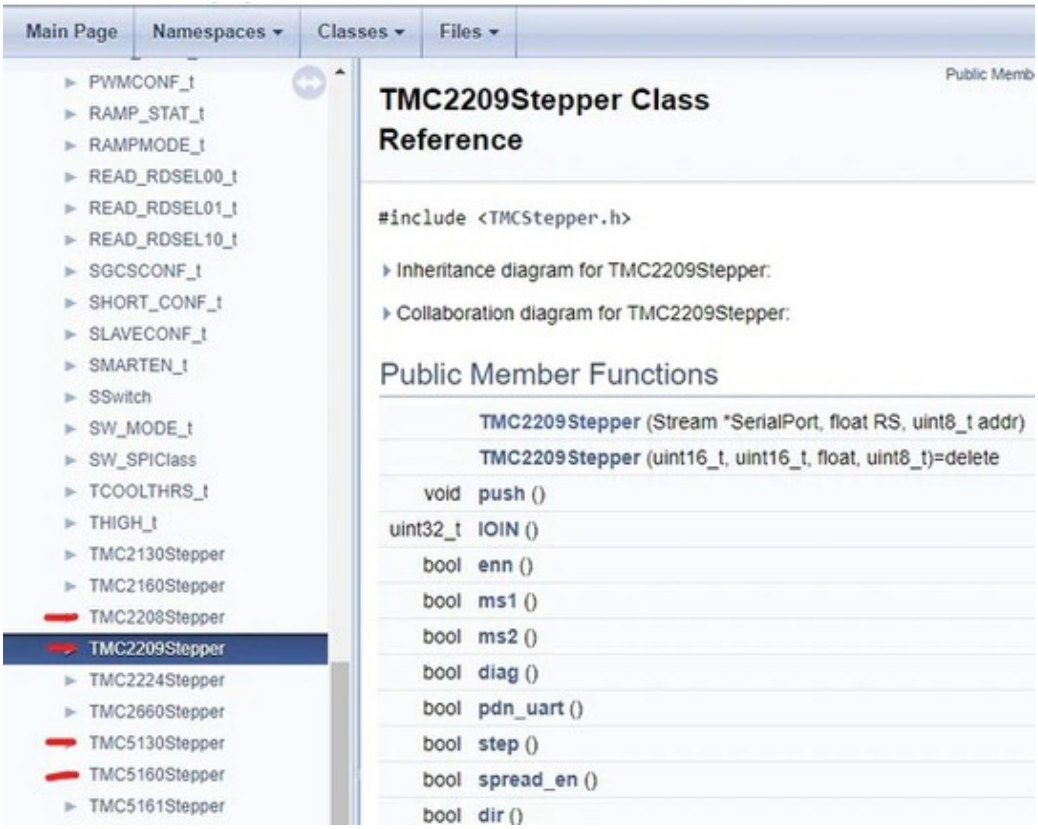
Then we place 0.12 (above in blue), this is the Ohms of the current sense resistors that are on your module. Refer to the Current Sense Resistor section below to learn more. This is critical to get right!

Then we place the slave address, which is 0. This is necessary because we can link many TMC2209s together if needed. In this case we are only using one.

Documentation

Let’s explore this documentation a bit more as you will need to reference it to change the values in the future and it's very good to understand how it works.

On the left side we can explore different classes, depending on the driver you have you can even look at the 2208, 2130, and more.



Let's say we want to change the current that the driver supplies to the motor. If we look at the Public Member Functions list for the TMC2209Stepper class, can you find something that says rms_current? (This stands for root mean square current, more on that later).

It doesn't exist here, so where is it?

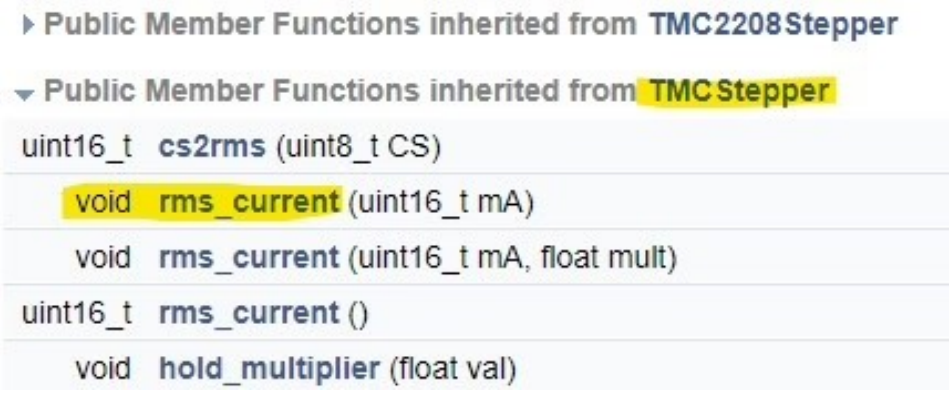
The TMC2209 is very similar to the TMC2208 as well as nearly every other Trinamic driver, so it inherits many of the functions from the TMC2208 as well as the universal Trinamic functions.

Click on the arrows that are in red below, notice that they reveal more functions that are inherited from TMC2208Stepper and TMC Stepper.

©2024 Daniel Frenkel



And here is the rms_current under the TMC Stepper section



In this example, we can see that to change the current of the motor to 500 mA, we need to write the following code in Arduino:

```
driver.rms_current(500);
```

This is because the library expects an integer for the milliamps of current.

Recommended Workflow for Setting Values: First, go to the TMC2209 datasheet and figure out what setting you want to change. Then go to the TMCStepper docs and try to find the name of the function that will control that value. Then click on it to figure out what values need to be passed into it.

FastAccelStepper

Did you know that we can drive the TMC2209 driver without any microcontroller? All we must do is apply 3.3V to the STEP pin and it will move one step. The stepper driver simply waits for a high signal on the step pin and when it sees a high pulse, it moves one step. Super simple!

The FastAccelStepper library is a set of algorithms that make controlling a motor very simple. We just tell it how far to move and how fast, and the library figures out how to pulse the step pin and sets the direction pin. This library will control the motor.

Let's begin by initializing FastAccelStepper:

```
FastAccelStepperEngine engine = FastAccelStepperEngine();
FastAccelStepper *stepper = NULL;
```

But it still needs to be initialized. For this "init" will be used.

```
engine.init();
stepper = engine.stepperConnectToPin(STEP_PIN);
```

FastAccelStepper vs AccelStepper

If you've ever looked for stepper motor library online, chances are that you've come across one called "AccelStepper" as it has been around for a long time and is very popular. It always works well, only nearly any Arduino processor. The reason it works universally is because it uses a process called "Bit Banging" to pulse the Step pin.

According to Wikipedia, bit banging is a "term of art" for any method of data transmission that employs software as a substitute for dedicated hardware to generate transmitted signals. In our case, AccelStepper ends up using the main processor to set ultra-fast pulses to the Step pin. This can cause problems because now we're tying up the main processor every time the motor moves.

However, there is a much better way to do this. The FastAccelStepper library makes use of two built-in peripheral modules that are available on the ESP32. In order to generate a pulse, it uses the Motor Control Pulse Width Modulator (MCPWM) module. And it also uses the Pulse Counter (PCNT) module to then count every pulse that it sends. Links to these modules are found on the resource page.

©2024 Daniel Frenkel

It's always a better idea to use dedicated hardware rather than tie up the processor when possible, which is why we use the FastAccelStepper library.

Dual Core Setup

Before we go further into FastAccelStepper, let's set up the dual core capability of the ESP32, because it may confuse you in the sketch. Here is the basic overview.

The ESP32 has two cores. For best performance, we need the motor functions to run on one core, and the WiFi functions to run on the other core.

By default, all Arduino code runs on Core 1 (including the WiFi processes). So, we will create Core 0. To create Core 0, we do this:

```
xTaskCreatePinnedToCore(
 MotorTask
, "MotorTask"
, 1024 * 4
, NULL
, 3
, NULL
, 0);
```

Next, let's create the function to run on this core, it needs to be named MotorTask because that's what we named it above.

Here we have our function running in a loop on core 0 only. This is where the motor will run, ensuring it only runs on Core 0. The while(true) creates an infinite loop.

```
void MotorTask(void *pvParameters) {
  while(true) {
```

Motor Setup

Let's continue setting up the motor in the setup() loop.

Start all the UART communications behind the scenes:

```
driver.Begin();
```

For operation with StealthChop, this parameter is not used, but it is required to enable the motor. In case of operation with StealthChop only, any setting is OK:

```
driver.toff(4);
```

Recommended blank time value:

```
driver.blank_time(24);
```

Disabled to use the external current sense resistors:

```
driver.I_scale_analog(false);
```

Use the external Current Sense Resistors. Do not use the internal resistor as it can't handle high current.

```
driver.internal_Rsense(false);
```

Ensure that the microstep resolution is selected by MSTEP register and not from the legacy pins:

```
driver.mstep_reg_select(true);
```

Set the current in milliamps:

```
driver.rms_current(current);
```

Set the stall value from 0-255. Higher value will make it stall quicker:

```
driver.SGTHRS(stall);
```

Set the number of microsteps. the TMC2209 uses MicroPlyer in order to turn every step into 256 microsteps. However, it’s not as accurate when you want to be precise. So set the microsteps as high as you can, and if your motor is not going as fast as you want it to, decrease the microsteps to 128, 64, 32, 16, 8, 4, 2. or 1. I like to keep 64 because it allows for a high top

```
driver.microsteps(motor_microsteps);
```

speed. However, your IC (ESP32) must be able to send a very fast pulse for higher microstepping:

Minimum speed at which point to turn on StallGuard. StallGuard does not work as very low speeds such as the beginning of acceleration, so we need to keep it off until it reaches a reliable speed:

```
driver.TCOOLTHRS(tcools);
```

Disable StealthChop PWM mode:

```
driver.TPWMTHRS(0);
```

Turn off smart current control, known as CoolStep. It's a neat feature but is more complex and messes with StallGuard:

```
driver.semin(0);
```

Set the shaft direction clockwise:

```
driver.shaft(true);
```

Disable SpreadCycle. We want StealthChop becuase it works with StallGuard:

```
driver.en_spreadCycle(false);
```

Enable UART control:

```
driver.pdn_disable(true);
```

Set the ESP32 pin that will toggle the direction of the motor stepper:

```
setDirectionPin(DIR_PIN);
```

Set the ESP32 pin that will enable and disable the motor driver. We usually want the driver disabled when it's not running. The downside of this is that the motor will spin freely. If you need it to hold the position, do not disable the driver:

```
stepper->setEnablePin(ENABLE_PIN);
```

The FastAccelStepper library has a neat "Auto Enable" feature that automatically enables and disables your board for you. When it needs to be moved it will enable the board, and when the movement is complete it will disable it. Again, when disabled the motor will spin freely:

```
stepper->setAutoEnable(true);
```

©2024 Daniel Frenkel

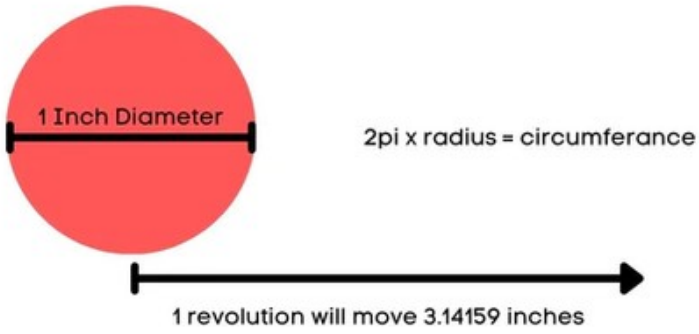
Understanding Positioning

When we tell the motor to move to position 5000 or 50000, what does that mean? How does that translate to a value we can understand? In the image below we have a blue object on a track. The left side is position 0 and the right side is position 5000. Whenever we move to a position, imagine your motor moving an object like this.



The value 5000 is based on step count. Remember the section about microsteps where we discussed how many steps are in one revolution.

When we tell the motor to move 5000 steps we are telling the motor how many microsteps to move. If we have 256 microsteps set it will require many more steps to move 1 meter than if we have 64 microsteps.



But using steps to move is very difficult to understand as a distance, so we need something we can understand like inches or centimeters. In this case I will work with inches which you can convert to different units.

Question: If we attach a rope or belt to the motor and want to move it just 1 inch, how many steps will we need to move?

First, that depends on your microstep setting! Second, it depends on the diameter of your pulley.

©2024 Daniel Frenkel

Assuming a 64 microstep setting and a 1-inch diameter pulley on the motor, to move 1 linear inch we need to move 4074.37 steps.

Since there are 200 steps per revolution and 64 microsteps per steps, that means each full revolution of the motor is 12,800 steps.

One full revolution of a 1 inch diameter pulley will move the pulley a total or 3.14159 linear inches. This can be figured out by converting radius to circumference.

2pi x 0.5 inches = 3.14159 inches

Now we divide the steps per revolution by 3.14159

12,800 / 3.14159 = 4,074 steps. This is the number of steps we need to command to move our belt one inch.

Starting a Movement

Let's look at our motor loop function.

When you see "stepper->" this means we are accessing the FastAccelStepper object and passing values to it. It will then control the stepper motor.

We begin by setting the current position of the motor. This happens right before the while(true) because we only want to set it one time. This can be done in the setup() loop as well

```
stepper->setCurrentPosition(0);
```

Next, we tell the motor to move TO a certain position. Remember, this library supports multiple types of movement commands:

1. move()
2. moveTo()
3. runForward()
4. runBackward()
5. applySpeedAcceleration()
6. moveByAcceleration()

I think that positioning moveTo() is the most important to understand, which is why this tutorial will use it. Here we tell it to move to the move_to_step position, which was set earlier to 3200 (3200 is an example, set it to whatever you want)

The ESP32 will now send 3200 HIGH pulses to the TMC2209 to get it to move 3200 steps.

```
stepper->moveTo(move_to_step);
```

While the motor starts moving to our new position, we need to wait. The while loop will now continually check if the motor is still moving. If it's moving, it will run the loop. Once the motor stops moving, the while loop will exit.

This loop does two things. First, it has a delay. The delay is there to just kill time while we wait for the motor to stop.

The second thing is we check if stallGuard has triggered a stall. The stall gets triggered in a separate interrupt that we created during setup using this code:

```
attachInterrupt(digitalPinToInterrupt(STALLGUARD_PIN),
stalled_position, RISING);
```

The TMC2209 will set the STALLGUARD_PIN to high when it detects a stall. By attaching an interrupt to this pin and monitoring for a rising edge, we can immediately detect a stall and react to it right away. The interrupt function sets a Boolean flag to true.

```
void IRAM_ATTR stalled_position() {
  stalled_motor = true;
}
```

In the if loop, we check if the variable stalled_motor == true. If it is, then we will react to it and stop the motor immediately. The forceStop() function will not decelerate, instead it will stop right away. After we stop the motor, we'll clear the stalled_motor flag.

After the stop, we wait two seconds (for fun) and then break from the loop and wait 3 more seconds (for more fun).

©2024 Daniel Frenkel

```
    while (stepper->isRunning() == true) {
      if (stalled_motor == true) {
        stepper->forceStop();
        Serial.println("Stalled");
        stalled_motor = false;
        delay(2000);
        break;
      }
      delay(1);
    }

delay(3000);
```

Finally, we move back to the home position 0 and repeat the entire process.

```
stepper->moveTo(0);
```

Advanced Features

StallGuard

Overview

Below is a screenshot of a video. In the video, a motor is pushing a window to close. While the window is closing, I place my hand at the very end. But rather than cutting my fingers off, the motor senses that something is in the way, and it stops it automatically, saving my fingers and ensuring that I live to type another day.



In this video it only takes five pounds of force for my fingers to stop the window. This works using a feature called StallGuard.

StallGuard works by monitoring the back EMF while driving the motor. As the motor encounters resistance, the back EMF of the motor begins to decrease, and the driver detects this. The driver then sends a signal to the ESP32 to stop the motor. To learn more, visit the StallGuard link on the resources page.

CRITICAL: The Arduino UNO and similar devices are too slow to see the stallGuard pin using digitalRead. You need to use interrupts for these devices. I recommend using an interrupt and not digitalRead.

When stallGuard is activated, the TMC2209 sends a HIGH signal on the Diag pin that is connected to pin 2 on the ESP32. We place an interrupt onto pin 2 which then runs some very fast code when it detects a stall.

Let's look at the interrupt function what will get triggered by this high pin.

```
void IRAM_ATTR stalled_position()
{
stalled_motor = true;
}
```

Notice the IRAM_ATTR. This is a special ESP32 word that makes sure to store this function in a special area of the memory, this way it is called very fast. We always use it for interrupt functions.

When this interrupt is called, we want to set the stalled_motor Boolean to true.

We now need to place some code to initialize this interrupt. By looking for a rising edge, we can see it very quickly and respond right away in the code. Remember, when a stall takes place, pin 2 is normally LOW, and then becomes HIGH. We want to watch for the rising edge when it goes from low to high to trigger this interrupt. We initialize it on the RISING edge like this:

```
attachInterrupt(STALLGUARD, stalled_position, RISING);
```

StallGuard Setup

We will now cover the basics of setting up StallGuard. But before we begin, you need to understand the difference between StallGuard2 and StallGuard4. StallGuard2 has been around for many years and is used on most of the older Trinamic drivers such as the TMC2208, TMC2130, TMC2160 and more. StallGuard4 is newer and is found on the TMC2209 and a couple other newer chips.

StallGuard4 is not better, it is different.

Go to page 6 of the datasheet, section 1.4. This is important to understand as we will set this in the settings. Here is how I best describe it:

Most Trinamic drivers give you the choice of using one of two chopper algorithms called SpreadCycle and StealthChop. SpreadCycle is super fast but makes the motor loud. StealthChop is near silent but can only be used at slower speeds. By default StealthChop is enabled for the TMC2209. To enable StealthChop please look at bit 2 of the GCONF register.

StallGuard2 ONLY works with SpreadCycle while StallGuard4 ONLY works with StealthChop.

The TMC2209 only has StallGuard4 which means we can only run it with StealthChop and it will not work at high speeds.

Setting StallGuard Values

Now that we have the interrupt set up in the code, we need to set the correct value to trigger StallGuard. There are 256 values we can set (0-255):

```
driver.SGTHRS(set_stall
```

Remember, the driver monitors the back EMF and triggers a stall when the back EMF reaches a threshold that we set. So, we need to tell it what that threshold point is, so it triggers correctly.

If the value is too low, it will never trigger a stall. But if the value is too high, the motor will immediately stall.

Go to page 55 of the TMC2209 datasheet.

SGTHRS is the value that we set that tells the driver when to trigger a stall. SGTHRS is based on the SG_RESULT result value. **SGTHRS is half of the SG_RESULT**.

SG_RESULT is a value between 0-510 that you can always monitor using UART. The great thing about it is that we can visually see when a stall will occur and then set the SGTHRS value to match up with SG_RESULT.

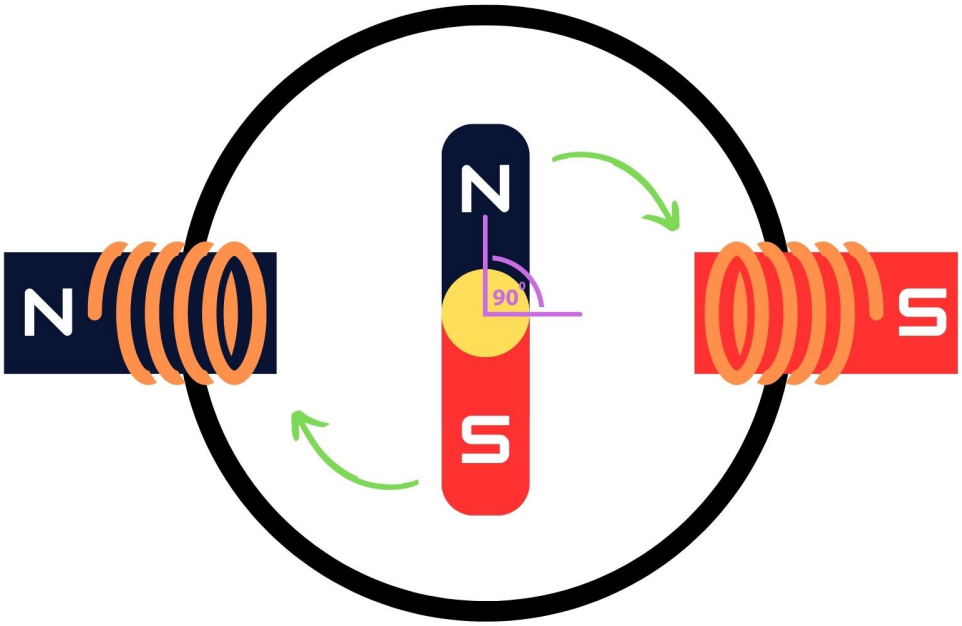
To monitor it, find the SG_RESULT register and use it to output the value to the serial monitor. This will allow us to monitor SG_RESULT and therefore allow us to set the perfect SGTHRS value.

```
Serial.print(driver.SG_RESULT);
```

How SG_RESULT works

Below is an extremely simplified diagram of a stepper motor. In the center we have the rotor which is just a permanent magnet with South on one end and North on the other. When the motor wants the rotor to move, the coils become energized and the N pole on the magnet is attracted to the S coil, and the S is attracted to the N coil.

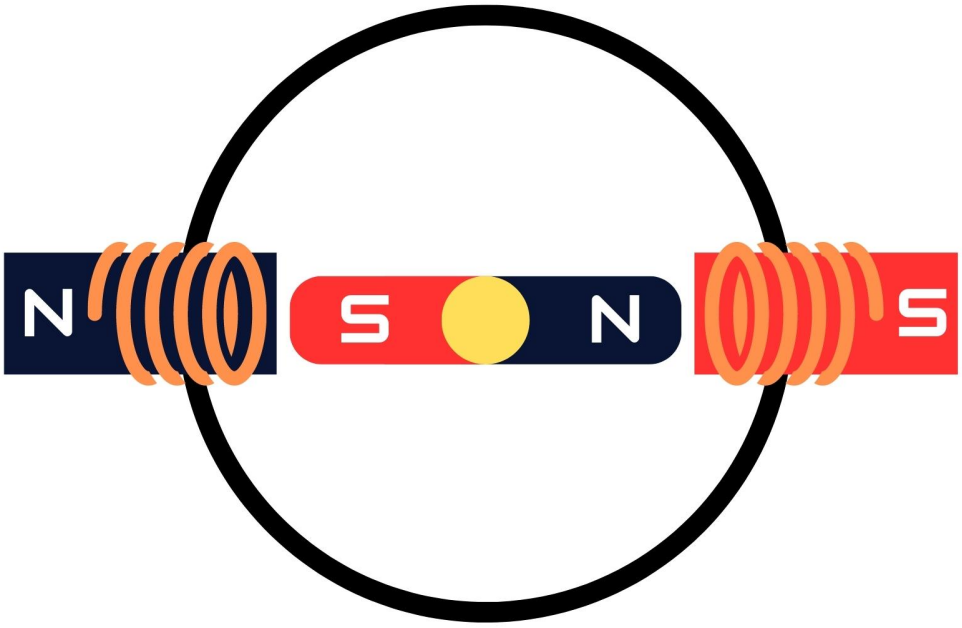
In a stepper motor, we always have maximum torque when the rotor is 90 degrees to the magnetic field generated by the stator, like in the image below. An actual stepper motor is more complex than this, but the principle is the same and very important to understand for StallGuard.



While the rotor is spinning, it is generating a voltage in the opposite direction. Remember that a motor can also be used as a generator. For example, a hydroelectric dam is simply a motor that is forced to spin by falling water and generates a voltage.

When we apply a voltage to a motor, it begins to spin and starts to act like a generator and creates voltage that pushes against our voltage.

What happens when the rotor stops spinning and stalls? The motor will look like the image below. In this case, the poles are all aligned, and no torque is being generated. There will also be no back EMF generated because the rotor is not spinning. When this happens, a stall occurs.



After years of using StallGuard, here is what you need to know. StallGuard is a combination of your:

1. **Current**
2. **Velocity**
3. **Acceleration**
4. **TCOOLTHRS setting**
5. **SGTHRS setting**

Current:

The current you are using will determine the amount of back EMF created, so first set your current to the level you need for your application. A high current will make the motor louder and stronger. A low current will do the opposite. Dial that in first.

Velocity:

Next, set the velocity. How fast do you need your motor to go.

Acceleration:

Next, set the acceleration. StallGuard has a very big downside which is very slow speeds are not reliable for StallGuard. This is because back-EMF is not generated while the motor is in standstill. And not enough back-EMF is generated during the initial acceleration, therefore stallGuard will always trigger during an acceleration. In fact, the driver will most likely not detect any stall during acceleration, until the max speed is reached, so a very long and slow acceleration is not good. Try to accelerate as fast as you can based on your project. Just know during the acceleration phase, StallGuard may not work, so can you risk having it not work until it gets to a stable velocity?

©2024 Daniel Frenkel

In my window opener, I cannot risk it, because what if the window is locked when trying to open the window? Damage may occur. I need to accelerate as quickly as possible to make sure StallGuard works, but not too fast because the window is heavy and cannot move without some acceleration. Take some time to get your acceleration right.

TCOOLTHRS setting:

Now we can set the TCOOLTHRS value. It is on page 26 of the datasheet.

This is the velocity threshold at which point StallGuard will turn on. Remember, StallGuard is not reliable at low speeds so Trinamic built this feature that automatically turns StallGuard off until a certain speed is reached. StallGuard becomes enabled when exceeding TCOOLTHRS velocity. It becomes disabled again once the velocity falls below this threshold.

We need to set the correct value, so let's look at the datasheet on page 26 to see how to set it.

This value can be difficult to understand because we are working with two different velocities. The velocity we have been working with thus far is the one that is set in Arduino and managed by the FastAccelStepper library. However, the TMC2209 does not understand what that means. Instead, it measures velocity in a different way.

Notice this statement in the datasheet: TCOOLTHRS ≥ TSTEP. Let's go to page 25 to the 0x12 register for TSTEP. This register outputs the velocity. What we need to do is find out the velocity that we are using normally. To do this, place this code into our main loop so we can check what velocity the motor is moving at.

```
Serial.println(driver.TSTEP);
```

This value is the actual measured time between two 1/256 microsteps. When the motor moves slower, the value will increase because the time between two microsteps gets longer. When the motor moves faster, the value will decrease.

Get your motor to operating velocity and check what TSTEP value you get. Notice that as the motor increases in speed, the value gets smaller. Set your TCOOLTHRS value to a number that is larger than the TSTEP value. For example, if normal operating velocity gives you a TSTEP of 300, set TCOOLTHRS to 400 or so. This means that when the motor velocity reaches 400, then StallGuard will turn on. Any slower than 400 and StallGuard will remain off to not give a false positive signal.

NOTE: Remember that you cannot run Serial print and the STEP pin at the same time on one processor because Serial print will slow down the STEP pulse and you will not get an accurate value. My workaround on the ESP32 (which is dual core) is to run the motor code on one processor and the print code on the other processor. If you have two Arduinos, you can also use one to generate the STEP pulse and the other to monitor the Serial output.

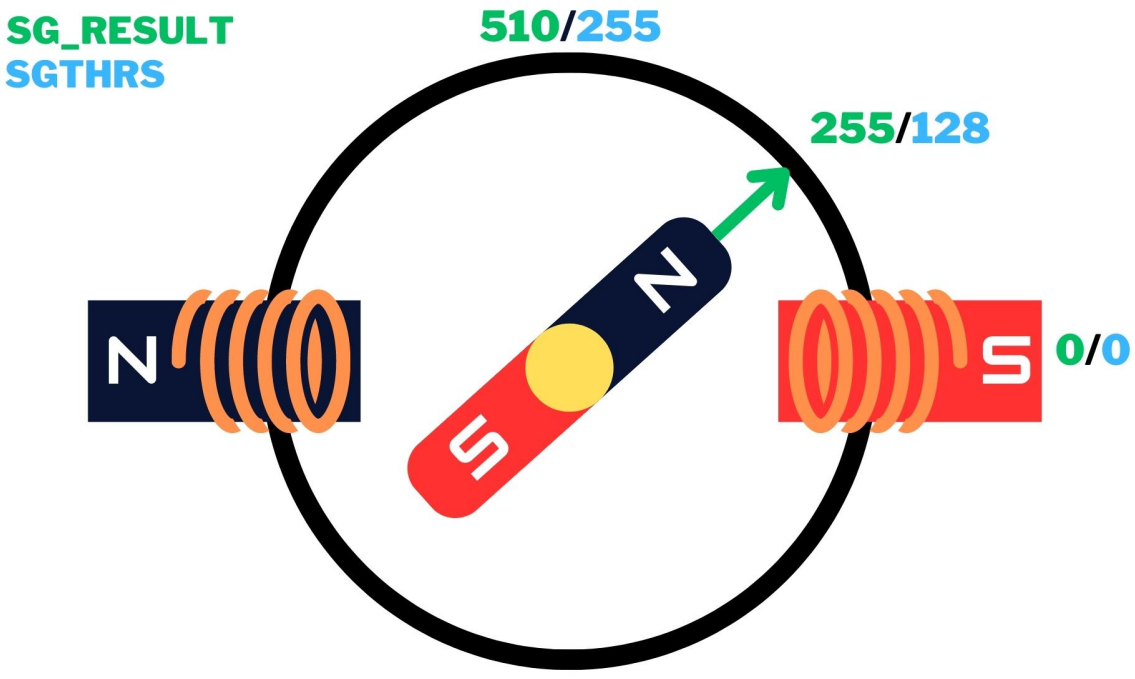
You can also use the built in pulse generator and free up your IC for the print command. To use the built in pulse generator, go to page 25 of the datasheet under VACTUAL. Set VACTUAL to a velocity that you would like by using the following code, be sure to replace VELOCITY with whatever works best:

```
driver.VACTUAL(VELOCITY);
```

SGTHRS setting:

Finally, we need to set the SGTHRS value which is the main value to set. It is a value from 0-255 as seen on page 55. A value of 0 will turn StallGuard off completely. The higher you go, the more sensitive it will be to tripping. On page 55 we can see that **the double** of this value is compared to SG_RESULT. The stall output becomes active if SG_RESULT falls below this value.

Here is a visual to help you understand. The optimal angle for the magnet to be from the coil is always 90 degrees, which will give us a SG_RESULT value of 510. When no force is applied to the motor, the driver will always maintain the 90-degree angle as it spins the motor.



However, as force is applied to the rotor (for example, if you squeeze the motor shaft with your fingers), it will not be able to maintain the 90-degree angle and will begin to approach 0, which is considered a stall. In the example above, if we read SG_RESULT we will get a value of 255. To

set **SGTHRS** to trigger a stall when the rotor is in this position, we set the value to 255/2 = 127.5 (But we need a whole number so let's round to 128).

Setting the SGTHRS value to 255 will always trigger a stall. The smaller the number, the more torque is required to trigger a stall. It is up to you to figure out exactly what value you'd like the stall to trigger at.

One way to find the best **SG_RESULT** and **SGTHRS** value is to run the code below in your sketch. Squeeze the motor with your fingers and watch the value change. Use it to set your stall value.

```
Serial.println(driver.SG_RESULT);
```

Recommended Workflow for Setting StallGuard

1. First, I figure out the current to set. I need to have enough current to move whatever needs to be moved, but not much more. If the current is too high, StallGuard will require a lot of force to trip, so don't set it much higher than the application calls for.
2. Then I figure out what velocity I want and set that.
3. Then I figure out what acceleration I want, trying to use the fastest I can for the application.
4. Then I set the TCOOLTHRS value by monitoring the TSTEP register. It's best to set TCOOLTHRS as close as you can to the TSETP value when operating at normal velocity. Remember TCOOLTHRS needs to be larger than TSTEP.
5. Then I start setting SGTHRS at 0 and start moving the motor and applying pressure to it with my fingers. I slowly increase the SGTHRS number until it stalls. Just keep running the motor, applying pressure, and increasing the value. Alternatively, you can monitor SG_RESULT and set the SGTHRS to half of the value where you would like the stall to trigger.

CoolStep

CoolStep is a cool feature that allows your motor to run cooler and more similar to a DC motor. This way the current automatically changes as the load on the motor changes. The downside is that you won't be able to use StallGuard if you use CoolStep.

Normally, if you set the current to 2000 mA, the motor will always run at that current, even if there is no load on it. That is wasteful and only generates heat when not required. CoolStep will simply monitor the SG_RESULT value and as it approaches 0, which means a stall is about to occur, it will increase the current to the motor. When the SG_RESULT value stays at 510 for a while, it will then decrease the current.

One great application for this is dynamic loads. Say that you need to move something that is always changing, like a window or door. You can use this to always run at the maximum current without overheating your device.

Simply set your current value to the max you are willing to allow it to go. Then monitor the CS_ACTUAL value to see how the current gets adjusted. You will also need to set the SEMIN and SEMAX values which set the points where the TMC will automatically increase and decrease the current when the SG_RESULT gets to the values. I will go into further detail on setting this up in a future book update. Please reach out to me for more information if you require it.

Finite State Machine

The proper way to run most motorized projects is with a finite state machine. If you're not familiar with state machines, you should be. You can learn more here.

My code "dual_core_state_machine.ino" has several states that automatically change from one to the next. You can add your own buttons, lights and sounds to each state. If you need more help, please contact me. I have provided the code which I think is fairly clear.

Single-Core Operation

The problem with using a single-core ESP32 with a stepper motor is that moving a motor by pulsing the STEP pin is a full-time job. The ESP32 is completely tied up when operating the stepper motor that it doesn't have any processing time left to do other tasks. Sometimes it will crash as well when the watchdog timer gets triggered because other tasks are not able to run.

This is not a problem if you have a simple device that does not need to complete other tasks, like WiFi updates. However, if you want to use WiFi or run LED lights, or anything else whatsoever while moving the motor, you'll be out of luck.

Fortunately, there's a solution. The TMC2209 contains an internal pulse generator that will drive the STEP pin for us. All we need to do is tell it what speed to run at.

Let's open the example "single_core_simple.ino" to get started.

To start the pulse generator, all we do is tell the TMC2209 what velocity to move at, and it will begin to generate pulses at that velocity. The datasheet outlines how to communicate velocity in section 14:

©2024 Daniel Frenkel

PARAMETER VS. UNITS		
Parameter / Symbol	**Unit**	**calculation / description / comment**
f_{CLK}[Hz]	[Hz]	clock frequency of the TMC2209 in [Hz]
µstep velocity v[Hz]	µsteps / s	v[Hz] = *VACTUAL*[2209] * (f_{CLK}[Hz] / 2^24) With internal oscillator: v[Hz] = *VACTUAL*[2209] * 0.715Hz

The microstep velocity in Hz, meaning steps per second, can be calculated using the formula in the box on the right. So, if we are using a microstep resolution of 16 microsteps per step, and have a motor that has 200 full steps per revolution, what velocity do we set to move at one revolution per second?

In this guide, we are using the internal oscillator, so the answer would be 3600 = VACTUAL * 0.715. To solve for VACTUAL, we rearrange the formula to be VACTUAL = 3600 / 0.715.

This means that in Arduino, we must set the VACTUAL value to 5035 to move at 3600 microsteps per second, giving us one revolution per second.

To start a movement, all we do is send this via UART:

```
driver.VACTUAL(5032);
```

This will immediately set the velocity to this value and the motor will begin to spin. However, what if we can to know how far the motor has moved? To monitor the steps that the TMC2209 has taken, we just need to monitor the index pin.

The index signal is a pulse from the TMC2209 that tells us when it has taken a step. We need to count these pulses with an interrupt, as outlined in the datasheet :

Hint
To monitor internal step pulse execution, program the INDEX output to provide step pulses (*GCONF.index_step*). It toggles upon each step. Use a timer input on your CPU to count pulses. Alternatively, regularly poll *MSCNT* to grasp steps done in the previous polling interval. It wraps around from 1023 to 0.

To set up the index pin, we need to make an additional connection. The index pin is a pin that's on the TMC2209 that will output the index signal, so be sure it is connected with a jumper wire.

```
#define INDEX_PIN 2
```

We need to turn the index pin on according to the datasheet.

5	*index_step* 0: INDEX output as selected by *index_otpw* 1: INDEX output shows step pulses from internal pulse generator (toggle upon each step)

We do it in the code here:

```
driver.index_step(true);
```

To monitor the pulses from the index pin, we'll attach an interrupt on the rising edge:

```
attachInterrupt(digitalPinToInterrupt(INDEX_PIN),
index_interrupt, RISING);
```

We'll run this ISR every time the index pin pulses:

```
void IRAM_ATTR index_interrupt() {
  if (move_to_max == true) {
    tmc_steps++;
  } else {
    tmc_steps--;
  }
}
```

The above code will allow us to track the number of steps that the internal pulse generator has taken. This way we can implement our own positioning function. We can use the isClosing variable and set it to true or false depending on the direction the motor is moving to add or subtract steps.

NOTE: The index output gives one pulse per electrical rotation, i.e., one pulse per each four fullsteps. Keep this in mind when writing position tracking code.

Acceleration

VACTUAL simply sets a speed for the motor. If you need the motor to accelerate, I recommend creating a for loop. Adjust the delay and the loop increment to change the rate of acceleration.

```
  for (int i = 0; i < vactual_velocity; i = i + 10) {
    driver.VACTUAL(i);
    delay(20);
    Serial.println(tmc_steps);
  }
```

We'll then use a while loop that waits for the motor to reach its position based on the INDEX count from the interrupt. Once the position is reached, the while loop will exit.

```
while (tmc_steps < move_to) {
delay(1);
}
```

If stallGuard is required, you can implement is like there to stop the motor once stall is detected. This stops the motor on stall and starts it after two seconds. Be sure to tune your TSTEP and SGTHRS values to get them just right.

Conclusion

I hope you've enjoyed learning a bit about Trinamic stepper motor drivers. It's been difficult to figure out how my info to put into this book and how much to leave out. But at this point you should have enough info to start writing basic firmware that can utilize a wide range of Trinamic features.

If you think something else should be included in this book, please email me at daniel@valarsystems.comand I will be happy to consider it. Thanks for learning!